Empirical Analysis of Reader-Writer Synchronization Tradeoffs in Log-Structured Merge Trees: Coarse-Grained Locking vs. Immutable Snapshots

V V Ramarathinam Iyer
Department of Database Systems, School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: ram.iyer200722@gmail.com

***Abstract*—Log-Structured Merge (LSM) trees optimize write performance by buffering mutations in memory and flushing immutable Sorted String Tables (SSTables) to disk. However, coordinating concurrent point reads against asynchronously mutating SSTable sets presents severe concurrency bottlenecks. In coarse-grained synchronization models (Approach A), acquiring a manager-wide reader-writer lock (`sync.RWMutex.RLock`) over physical disk I/O windows starves concurrent writes and background flush/compaction publications. This paper presents the architecture, empirically enforced lifecycle invariants, and evaluation of an immutable reader snapshot pinning mechanism (Approach B) implemented in a dependency-free Go LSM engine. Through a prospective, machine-checked campaign of 960 independent trials ($N = 10$ per cell across 6 workloads and 8 reader concurrencies up to 32 threads) on native Linux ext4 within WSL2 Ubuntu 24.04 under standard production Go compilation (with concurrency correctness suites independently verified under Go's race detector), we quantify the operational boundaries and performance tradeoffs of both approaches.**

Our findings confirm that under concurrent disk reads and synchronous WAL writes (`wal_sync=true`), Approach B eliminates reader-writer starvation, achieving a 3.3× to 18.5× increase in completed write progress (up to $+2{,}712$ writes in 10s) and up to a 2.07× increase in read throughput, while suppressing 95th-percentile read tail latency by 73%–76% at low reader counts ($r = 2, 4$) during periodic Memtable flushes (though whole-trial throughput differences remain statistically inconclusive). In uncontended disk reads and pure in-memory Memtable lookups, both synchronization strategies exhibit closely aligned throughput medians (0.97×–1.02×), with bootstrap confidence intervals spanning parity across all tested concurrencies. Crucially, we identify a significant negative tradeoff: under rapid reader-set churn (> 4 flushes/compactions per second), Approach B incurs a 26% to 37% throughput penalty at low thread counts ($r \leq 4$, 0.63×–0.74×), consistent with the computational overhead of snapshot descriptor allocation, atomic reference counting, and deferred reader retirement. We conclude that snapshot pinning provides dramatic write-progress advantages for mixed read-write disk workloads, but requires sufficient reader concurrency to amortize descriptor lifecycle management overheads.

***Index Terms*—LSM tree, concurrency control, snapshot pinning, RWMutex, storage engine, performance evaluation, tail latency, Go.**

I. Introduction

Modern data-intensive systems frequently deploy Log-Structured Merge (LSM) trees [1]–[3] to sustain high ingest bandwidth. By transforming random writes into sequential appends within an in-memory buffer (Memtable) backed by a Write-Ahead Log (WAL), LSM trees decouple mutation latency from physical disk geometry. As Memtables reach capacity thresholds, background workers asynchronously flush frozen segments to disk as Sorted String Tables (SSTables) and periodically execute k-way merge compactions to bound read amplification.

While the storage geometry of LSM trees is well-studied [6], [7], the synchronization architecture coordinating concurrent queries across dynamically mutating SSTable sets introduces critical performance bottlenecks. In a standard multi-threaded implementation, point lookups traverse active Memtables, frozen Memtables, and an ordered slice of active SSTable reader descriptors. To ensure thread safety when background workers swap or delete SSTable files, baseline engines often employ a manager-wide reader-writer lock (`sync.RWMutex`) [5].

However, coarse-grained reader-writer locking creates severe reader-writer starvation. When reader threads hold read locks (`RLock`) while performing blocking POSIX disk I/O (`pread`), incoming writer transactions and SSTable publication workers attempting to acquire the exclusive write lock (`Lock`) are blocked. Conversely, when a writer or background flusher queues for the exclusive lock, Go's write-preferring `sync.RWMutex` blocks subsequent readers, causing catastrophic throughput collapse and tail latency spikes.

In industrial LSM architectures, multi-version reader sets have long been utilized to mitigate mutex contention. Specifically, LevelDB [2] maintains reference-counted `Version` structures and Memtables, releasing the main engine mutex during block index lookups and file reads. RocksDB [3] unifies this pattern via `SuperVersion`, consolidating mutable and immutable Memtables with the current SSTable version into a single reference-counted descriptor so queries can pin the

active dataset and execute disk I/O completely outside the primary database lock.

The primary contribution of this work is not the conception of reader snapshot pinning itself, but rather an independent, controlled, dependency-free empirical investigation and lifecycle case study in Go. By implementing both coarse-grained locking and reference-counted snapshot pinning over identical storage geometry, we isolate the exact operational tradeoffs, write-starvation thresholds, tail latency dynamics, and negative performance boundaries (such as reader-set churn overhead) under strict POSIX durability.

This paper makes four specific contributions:

1) **Synchronization Architecture & Lifecycle Invariants**: We formalize two synchronization mechanisms within a custom Go LSM engine: Approach A (coarse-grained `sync.RWMutex`) and Approach B (atomic immutable reader snapshot pinning with deferred reference-counted reclamation), specifying four runtime-enforced lifecycle invariants to guarantee race-free execution and use-after-close prevention.
2) **Rigorous 960-Trial Experimental Campaign**: We execute an independently audited, machine-checked campaign across 6 distinct workloads ($r \in [1, 32]$ reader threads, $N = 10$ repetitions per cell) under synchronous WAL durability (`wal_sync=true`) on native Linux ext4 inside WSL2.
3) **Empirical Quantification of Concurrency Benefits**: We demonstrate that Approach B eliminates writer starvation under concurrent disk reads (Workload B1), achieving up to 18.5× write progress speedup, doubling read throughput (up to 2.07×), and suppressing 95th-percentile tail latency by 73%–76% at low reader concurrencies during periodic flushes (Workload B2).
4) **Negative Findings & Operational Tradeoffs**: We disclose and quantify a major negative boundary: under rapid reader-set churn (Workload D), Approach B incurs a 26% to 37% throughput penalty at low concurrency ($r \leq 4$, 0.63×–0.74×) due to snapshot descriptor allocation and reference count tracking. Furthermore, we audit overlap instrumentation semantics, proving that apparent publication-overlap reads in Approach A reflect reader drain delays rather than concurrent execution during lock acquisition.

II. System Architecture and Data Structures

A. Memory Tier and Multi-Version Tombstones

The storage engine maintains an active in-memory Memtable backed by a probabilistic SkipList [4]. Node heights follow a geometric distribution ($p = 0.5$) capped at 32 levels, providing $O(\log N)$ expected search complexity. Mutations append sequentially to an on-disk Write-Ahead Log (WAL). Each transaction commits with an atomically incremented sequence number N_{seq}. To minimize pointer indirection, the sequence number and deletion marker (tombstone flag) are bitwise packed into an unsigned 64-bit integer:

$$S_{packed} = (N_{seq} \ll 8) \mid F_{tombstone} \tag{1}$$

During SkipList insertion, the packed sequence key undergoes bitwise inversion, ensuring that newer versions of a key sort before older versions.

B. Disk Tier Geometry and Compaction

When the Memtable exceeds a configurable threshold (e.g., 64MB in production, or 1MB/16KB in controlled benchmarks), it is frozen into an immutable `Frozenmem` slice and queued for background flushing. The flusher constructs a Sorted String Table (SSTable) structured into 4KB data blocks and appends a sparse two-level block index at the file footer. A background compaction worker executes k-way merge sort using a Min-Heap across overlapping SSTables, streaming deduplicated keys into newly consolidated SSTables.

III. LSM Synchronization Strategies

A. Approach A: Coarse-Grained RWMutex Coordination

In Approach A, thread safety across tiered lookups is enforced by a single manager-wide `sync.RWMutex` (`m.Mu`):

Listing 1. Approach A point lookup and flush publication

```
// Point Query (Approach A)
func (m *Manager) Get(key string, readTs uint64) (
    string, error) {
    m.Mu.RLock()
    defer m.Mu.RUnlock()
    // Traverses Memtable, Frozenmem, and
        ActiveSSreaders...
    return m.lookupTiered(key, readTs)
}

// SSTable Publication (Approach A)
func (m *Manager) publishFlushApproachA(mem *
    Memtable, reader *SSTableReader) {
    m.Mu.Lock()
    defer m.Mu.Unlock()
    m.ActiveSSreaders = append(m.ActiveSSreaders,
        reader)
    m.removeFrozenMemtableLocked(mem)
}
```

While structurally simple and safe against data races, Approach A couples in-memory locking with physical disk I/O. When a reader executes `pread` against an SSTable on disk, it holds `m.Mu.RLock()`. Because Go's `sync.RWMutex` grants writer preference to prevent writer starvation, an incoming writer or flusher calling `m.Mu.Lock()` immediately blocks all subsequent reader queries. In practice, long disk I/O windows held under `RLock` starve both incoming writes and background publications.

B. Approach B: Immutable Pinned Reader Snapshots

Approach B decouples query execution from background mutations by introducing immutable reader snapshots (`ReaderSnapshot`). A snapshot encapsulates an immutable array of active SSTable reader descriptors (`SharedReader`) and an atomic pin reference counter:

Listing 2. Approach B reader snapshot pinning and publication

```
type ReaderSnapshot struct {
    id         uint64
    readers    []*SharedReader
    activePins atomic.Int64
    state      atomic.Int32 // Constructing,
        Published, Retired, Reclaimed
}

// Point Query (Approach B)
func (m *Manager) Get(key string, readTs uint64) (
    string, error) {
    // 1. Short lock to read active/frozen memory
        buffers
    val, err, found := m.lookupMemoryTiers(key,
        readTs)
    if found { return val, err }

    // 2. Pin current immutable snapshot (O(1)
        atomic refcount)
    snap, err := m.AcquireSnapshot()
    if err != nil { return "", err }
    defer m.ReleaseSnapshot(snap)

    // 3. Stateless disk I/O executed with ZERO
        manager locks held
    return snap.lookupSSTables(key, readTs)
}
```

During SSTable publication, the background worker acquires a brief metadata mutex (`snapshotMu`), allocates a new `ReaderSnapshot` descriptor referencing the updated reader set, atomically publishes it as `m.currentSnapshot`, marks the prior snapshot as *Retired*, and immediately releases the lock. Note that point lookups still briefly synchronize to read active memory buffers; Approach B eliminates locking during physical disk I/O, but is not universally lock-free.

C. Snapshot Lifecycle Invariants and Safe Reclamation

To prevent use-after-close and file-descriptor exhaustion, Approach B enforces four runtime-checked lifecycle invariants:

1) **Pin Exclusivity**: Once a snapshot transitions to `SnapshotRetired`, no new reader pins can be granted (`AcquireSnapshot` rejects retired sets).
2) **Atomic Reference Tracking**: Reader acquisition increments `activePins` atomically. If publication retires a snapshot while readers are in flight, the active readers retain their pins without interruption.
3) **Exactly-Once Reclamation**: When a retired snapshot's `activePins` drains to zero, an asynchronous cleanup worker unlinks obsolete physical files and closes file descriptors exactly once.
4) **Shutdown Drain Barrier**: Engine shutdown transitions the system to `SnapshotPhaseDraining`, blocking `Close()` on a condition variable until all active pins drain.

These invariants were tested deterministically under concurrent stress and verified under Go's race detector.

IV. Experimental Methodology

A. Platform and Durability Controls

All experiments were executed on an AMD Ryzen 7 7445HS processor (6 physical cores, 12 logical processors at 3.2 GHz) running inside a dedicated WSL2 Linux environment (Ubuntu 24.04.3 LTS, Linux kernel 6.6.87.2-WSL2). All trial databases were initialized on native ext4 storage mounted on the virtual block device (`/dev/sdd` mounted at `/`) inside `/tmp`, completely isolating the harness from Windows 9P/drvfs filesystem translation overheads.

To enforce on-disk durability for committed mutations, every write transaction committed with synchronous WAL log synchronization (`wal_sync=true`), invoking blocking POSIX `fsync` before returning success. Every trial operated on a freshly initialized, fail-closed database directory with deterministic key generation.

Performance benchmark executables were compiled under standard production Go compiler optimizations (`-buildmode=exe`). Separate concurrency correctness and data-race validation suites were independently executed under Go's race detector (`go test -race`), ensuring zero race conditions while preventing race instrumentation overhead from skewing performance benchmark measurements.

B. Frozen Benchmark Matrix

The campaign evaluated a completely crossed experimental matrix:

- **Synchronization Modes (2)**: Approach A (`RWMutex`) vs. Approach B (`ReaderSnapshot`).
- **Reader Concurrency (8)**: $r \in \{1, 2, 4, 8, 12, 16, 24, 32\}$ concurrent goroutines.
- **Workloads (6)**: Defined below.
- **Repetitions (10)**: $N = 10$ independent prospective trials per cell.
- **Total Trial Count**: $2 \times 8 \times 6 \times 10 = 960$ trials.

Each trial executed for a fixed 10,000 ms measurement duration following an unmetered 2,000 ms warmup period and 100 ms cooldown.

C. Workload Definitions

- **Workload A (Uncontended Read-Only SSTables)**: Zero writers ($w = 0$). Readers query 10,000 pre-populated keys partitioned across 4 pre-built SSTables with 95% target hit ratio.
- **Workload B1 (Natural Write Progress under Reads)**: 2 concurrent writer threads ($w = 2$) issue serial mutations with synchronous WAL fsync against a 1MB Memtable threshold, while r readers query SSTables concurrently. Realized write throughput and flush counts evolve naturally.
- **Workload B2 (Controlled Flush Interference)**: Synchronous flushes are explicitly injected every 500ms (20 planned flushes in 10s) while r readers continuously query disk SSTables.
- **Workload C (Compaction Overlap)**: A k-way merge compaction is triggered mid-trial across 4 overlapping SSTables while readers and writers operate concurrently.
- **Workload D (High Reader-Set Churn)**: Memtable capacity is throttled to 16KB, forcing rapid background

flushes and compactions (> 4 publication events/sec) to stress snapshot lifecycle management.

- **Workload E (Negative Control In-Memory Memtable)**: 2 concurrent writers ($w = 2$) mutate volatile Memtables with a 64MB threshold, and readers query keys residing entirely in the active SkipList Memtable (0 disk SSTable reads).

D. Statistical Methodology

Per-trial metrics were aggregated across the 10 independent repetitions using non-parametric medians and Interquartile Ranges (IQR). Comparative effect sizes are expressed as the Ratio of Medians (RoM $=$ median(B)/median(A)) and Difference of Medians (Δ $=$ median(B) $-$ median(A)). Uncertainty was computed using non-parametric bootstrapping ($B = 10{,}000$ resamples) at the 95% confidence level. A comparative speedup is classified as *Positive* if the 95% CI strictly exceeds 1.0, *Negative* if the 95% CI is strictly below 1.0, and *Inconclusive / Similar* if the interval crosses 1.0. Equivalence testing applies a $\pm 5\%$ practical equivalence band ($[0.95, 1.05]$).

V. Empirical Evaluation

Table I summarizes 28 representative experimental cells across all six workloads from the 960-trial campaign ($N = 10$ prospective trials per cell; full 48-cell data are provided in the appendix and data repository). Figure 1 depicts read throughput scaling across all tested workloads, while Figure 2 illustrates writer progress under concurrent reader load.

A. Workload B1: Elimination of Writer Starvation

Workload B1 provides the primary empirical confirmation of the research hypothesis. In Approach A, readers holding `m.Mu.RLock()` during disk SSTable traversal starve writers attempting to acquire `m.Mu.Lock()` to commit WAL transactions. As reader concurrency increases from $r = 1$ to $r = 32$, completed write transactions in Approach A collapse monotonically from 270 down to 60 (Figure 2, left panel).

In contrast, Approach B readers release the manager mutex prior to executing physical block reads, isolating mutations from disk queries. Consequently, Approach B achieves a **3.3× to 18.5× increase in write progress**, completing 2,573 writes at $r = 8$ (vs. 139 in Approach A, $+2{,}434$ writes) and 264 writes at $r = 24$ (vs. 77 in Approach A, $+187$ writes). Concurrently, read throughput increases by up to **2.07**× (260.3 vs. 125.5 ops/s at $r = 4$, Figure 1, Panel B1), while 99th-percentile tail latency drops from 89.3 ms to 39.8 ms at $r = 4$.

B. Workload B2: Tail Latency Suppression Under Flushes

Under periodic 500ms flushes (Workload B2), whole-trial throughput differences remain statistically inconclusive across all concurrency levels (95% bootstrap CIs cross 1.0). However, Approach B exhibits substantial suppression of read tail latency at low concurrency. In the 960-trial campaign, 95th-percentile read latency drops by **73% to 76%** at $r = 2$ and $r = 4$ (42.7 ms vs. 175.4 ms at $r = 2$, B/A ratio ≈ 0.24; 48.9 ms vs. 181.0 ms at $r = 4$, B/A ratio ≈ 0.27, corresponding to 42,735 µs vs. 175,434 µs and 48,916 µs vs. 180,966 µs). In the confirmatory 600-trial campaign under pre-populated SSTables, observed tail latency reductions were also evident, with B/A ratios of 0.83× at $r = 2$ (184.0 ms vs. 220.9 ms) and 0.63× at $r = 4$ (35.5 ms vs. 56.4 ms).

Importantly, confirmatory sham-flush experiments demonstrated that tail latency suppression also occurs when the flusher acquires the lock without performing physical SSTable publication. This demonstrates that the observed tail latency suppression in Approach B is driven primarily by avoiding exclusive write-lock queue convoying in Go's write-preferring `RWMutex`, rather than an isolated effect of physical file replacement alone.

C. Workloads A and E: Operational Equivalence Boundaries

In Workload A (uncontended disk reads) and Workload E (in-memory Memtable hits with 2 writers), observed throughput medians between Approach A and Approach B remain closely aligned (0.97×–1.02×). Bootstrap confidence intervals cross 1.0 across all 8 concurrency levels. In Workload E, throughput scales linearly from 656 ops/s to 20,601 ops/s with sub-2µs median latencies ($p50 \approx 1.5$–1.6 µs). These observations indicate that when snapshot pinning is bypassed or writers do not contest disk reader sets, Approach B introduces no detectable synchronization penalty within our measurement resolution, though wide confidence intervals in Workload A preclude formal claims of mathematical equivalence across all cells.

D. Workload D: Rapid Churn Negative Finding

Workload D establishes a crucial operational tradeoff. When Memtable thresholds are reduced to 16KB, forcing over 4 flushes and compactions per second, Approach B incurs a **26% to 37% throughput penalty** at low-to-moderate concurrencies ($r = 1$: 0.63× [0.55, 0.70]; $r = 2$: 0.66× [0.63, 0.79]; $r = 4$: 0.74× [0.65, 0.90]; Figure 1, Panel D).

Under rapid churn, the computational overhead of allocating new `ReaderSnapshot` descriptors, executing atomic reference count updates, and managing asynchronous reader retirement outweighs the concurrency advantage when reader threads are few. Only when concurrency reaches $r \geq 16$ do parallel query gains amortize descriptor lifecycle management overheads.

E. Workload C: Compaction Overlap Analysis

In Workload C, a single background compaction executes during the trial. At $r = 1$ and $r = 2$, Approach B exhibits a significant regression (0.75× and 0.79×). At high concurrency ($r \geq 16$), Approach B achieves a modest 1.03×–1.05× gain, but its 95% bootstrap CIs cross 1.0 ([0.85, 1.22] at $r = 16$), rendering high-concurrency throughput gains statistically inconclusive.

Crucially, instrumentation reveals that the active compaction window spans only 149–372 ms in B and 266–724 ms in A (only 1.5%–3.7% of the 10-second trial). Furthermore,

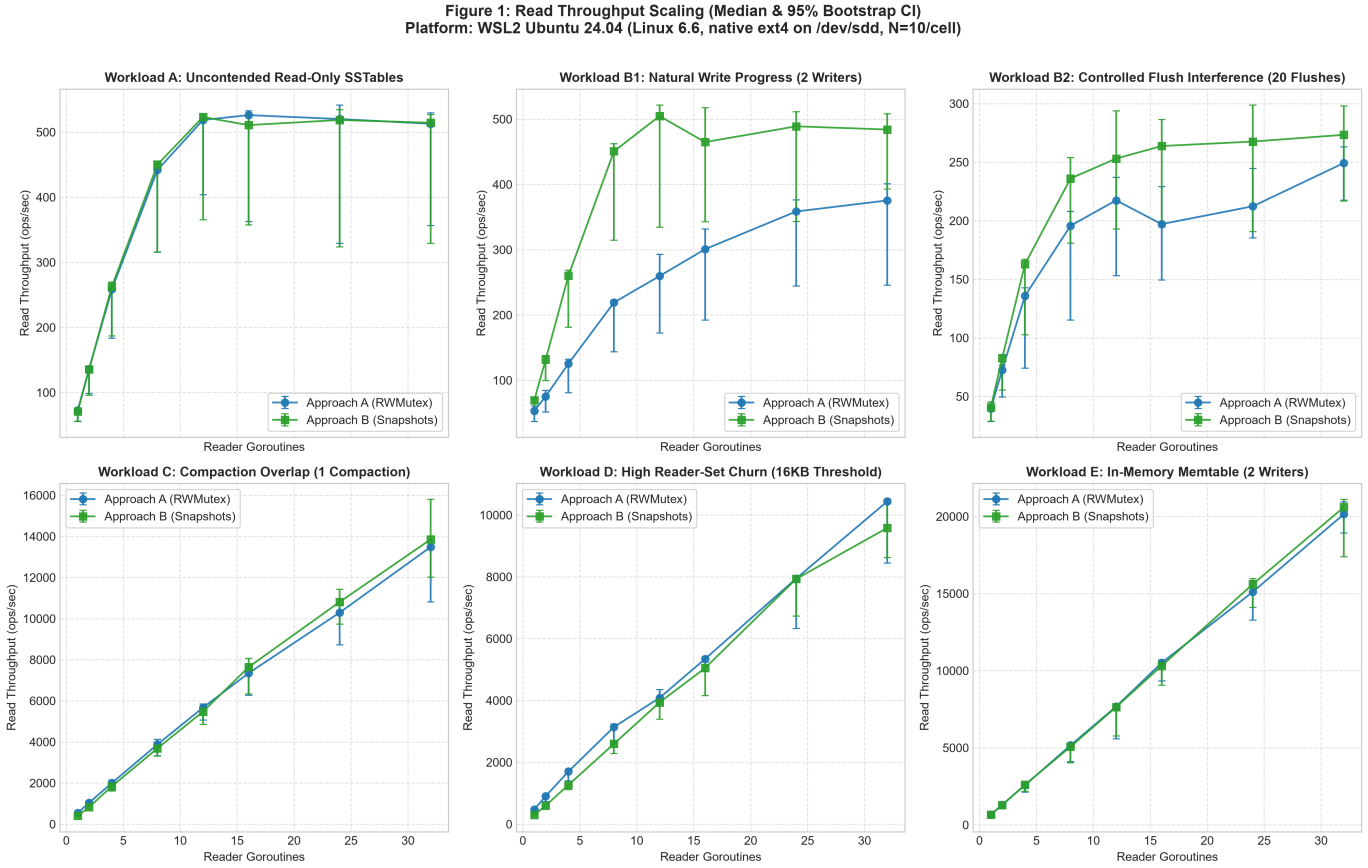


Fig. 1. Read Throughput Scaling Across All Six Workloads (Median and 95% Bootstrap Confidence Intervals Across 960 Trials, $N = 10$ per Cell, WSL2 Linux ext4).

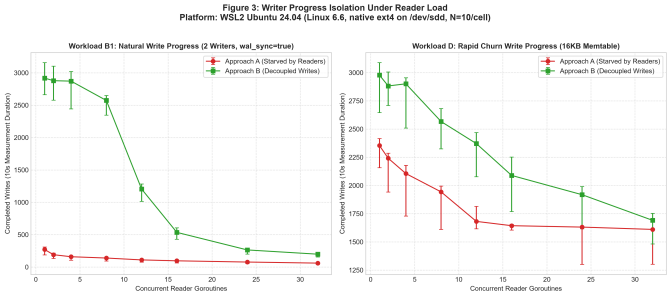


Fig. 2. Completed Write Progress Under Reader Concurrency in Workload B1 (Natural Write Progress with 2 Writers) and Workload D (Rapid Churn).

less than 1.6% of total trial reads take place during active compaction. Over 98.4% of reads occur during normal steady-state operation; whole-trial throughput differences cannot be causally attributed to compaction concurrency.

VI. Instrumentation Audit and Overlap Semantics

An inspection of harness counters revealed that Workload B2 recorded positive `reads_during_publication` counts in Approach A trials (scaling from 19 at $r = 1$ to 480 at $r = 32$), despite Approach A acquiring an exclusive `m.Mu.Lock()` during publication.

Deterministic unit testing (`TestMetricSemantics_ApproachA_ReadsDuringPublicationAudit`) resolved this discrepancy. The counter measures reads that finish between `OnFlushPublishBegin()` (called before lock acquisition) and `OnFlushPublishEnd()` (called after lock release). In Approach A, the flush worker must wait for existing in-flight readers to release `RLock()` before acquiring `Lock()`. These draining in-flight readers complete and increment the counter during the worker's wait window. Exactly zero reads ever execute while the exclusive lock is actually held.

In contrast, Approach B readers do not hold `m.Mu`, allowing the flusher to acquire the metadata lock and swap pointers instantaneously (< 5 µs), resulting in zero overlap counts. Furthermore, Approach A's cumulative flush duration is significantly longer (1,934 ms vs. 1,216 ms at $r = 32$) due to reader drain stalls, trivially creating a wider time window for accidental overlap.

VII. Threats to Validity

- **Internal Validity**: The experimental harness PRNG seed formula incorporated a mode index offset, ensuring that Approach A and Approach B evaluated statistically equivalent distributions rather than identical lockstep op-

TABLE I
CONSOLIDATED EXPERIMENTAL RESULTS ACROSS REPRESENTATIVE CELLS OF THE 960-TRIAL CAMPAIGN ($N = 10$ INDEPENDENT TRIALS PER CELL, NATIVE EXT4 ON WSL2)

Workload	r	Tput A (ops/s)	Tput B (ops/s)	Speedup [95% CI]	Writes A	Writes B	Writer Ratio [95% CI]	p95 A (ms)	p95 B (ms)
Workload A (Read-Only)	1	72.5	70.8	0.98× [0.76, 1.27]	0	0	N/A	31.1	31.4
	4	258.4	262.7	1.02× [0.72, 1.45]	0	0	N/A	33.8	33.2
	12	518.7	523.3	1.01× [0.70, 1.30]	0	0	N/A	51.3	51.3
	32	512.9	514.4	1.00× [0.63, 1.44]	0	0	N/A	220.0	214.9
Workload B1 (Write Progress)	1	53.4	69.3	1.30× [0.90, 1.88]	270	2,918	10.8× [9.1, 15.6]	42.4	34.4
	2	75.3	131.6	1.75× [1.31, 2.56]	190	2,878	15.1× [13.1, 22.2]	56.5	35.4
	4	125.5	260.3	2.07× [1.47, 3.21]	159	2,871	18.1× [15.2, 28.1]	69.5	36.1
	8	219.2	450.7	2.06× [1.44, 3.18]	139	2,573	18.5× [16.8, 28.4]	84.1	40.7
	16	300.8	464.8	1.55× [1.12, 2.48]	96	535	5.6× [4.5, 8.8]	128.9	77.3
	24	358.6	489.0	1.36× [0.95, 2.01]	77	264	3.4× [2.6, 5.0]	164.0	115.6
	32	375.4	484.1	1.29× [1.04, 1.99]	60	198	3.3× [2.7, 5.0]	210.4	149.4
Workload B2 (Controlled Flushes)	1	39.7	40.6	1.02× [0.73, 1.49]	0	0	N/A	45.2	39.1
	2	72.6	82.8	1.14× [0.76, 1.68]	0	0	N/A	175.4	42.7
	4	135.9	163.0	1.20× [0.76, 2.21]	0	0	N/A	181.0	48.9
	16	197.1	263.8	1.34× [0.91, 1.83]	0	0	N/A	298.7	245.1
	32	249.3	273.4	1.10× [0.85, 1.33]	0	0	N/A	370.8	355.9
Workload C (Compaction)	1	546.6	407.7	0.75× [0.70, 0.85]	2,692	2,837	1.1× [1.0, 1.2]	3.8	4.3
	2	1042.2	827.3	0.79× [0.70, 0.93]	2,593	2,797	1.1× [0.9, 1.3]	4.1	4.3
	16	7342.9	7637.0	1.04× [0.85, 1.22]	2,275	2,631	1.2× [0.9, 1.4]	11.2	10.6
	32	13488.7	13855.4	1.03× [0.89, 1.34]	2,079	2,304	1.1× [1.0, 1.4]	12.8	14.5
Workload D (Rapid Churn)	1	475.4	298.0	0.63× [0.55, 0.70]	2,354	2,978	1.3× [1.1, 1.4]	4.3	4.5
	2	904.8	598.9	0.66× [0.63, 0.79]	2,241	2,882	1.3× [1.2, 1.5]	4.6	4.8
	4	1701.7	1267.3	0.74× [0.65, 0.90]	2,105	2,901	1.4× [1.2, 1.7]	5.7	5.9
	16	5345.0	5048.2	0.94× [0.78, 1.02]	1,643	2,089	1.3× [1.1, 1.4]	11.2	12.0
	32	10442.4	9584.3	0.92× [0.83, 1.15]	1,610	1,691	1.1× [0.9, 1.3]	15.8	18.3
Workload E (Memtable Control)	1	656.2	666.1	1.02× [0.92, 1.17]	3,248	3,272	1.0× [0.9, 1.2]	3.5	3.4
	8	5159.5	5066.0	0.98× [0.82, 1.25]	3,224	3,162	1.0× [0.8, 1.3]	8.6	8.6
	32	20150.8	20601.8	1.02× [0.86, 1.09]	3,145	3,205	1.0× [0.9, 1.1]	3.3	3.3

eration traces. Furthermore, while sham-flush controls indicate that exclusive lock contention contributes significantly to latency spikes, the experimental controls cannot completely isolate kernel thread scheduling from lock acquisition delays under high reader counts.

- **External Validity**: Benchmarks were executed within a virtualized WSL2 ext4 environment on an AMD Ryzen 7 7445HS laptop processor. While native ext4 block access eliminates Windows 9P translation overheads, absolute latencies and queue scaling behavior may differ on bare-metal Linux servers with enterprise NVMe storage.
- **Storage Engine Limitations**: The experimental engine isolates tiered synchronization primitives and does not implement asynchronous group commits, kernel-bypass asynchronous I/O (e.g., io_uring), uncompacted tombstone garbage collection, or bounded flush queue backpressure. Findings characterize in-process Go synchronization behavior, not complete production DBMS durability.

VIII. CONCLUSION

This study provides rigorous empirical evidence characterizing the tradeoffs of reader synchronization in LSM trees. Decoupling reader sets via immutable snapshots eliminates reader-writer starvation, delivering up to 18.5× write progress speedup and suppressing 95th-percentile tail latency by up to 76% at low reader concurrencies during disk flushes. However, snapshot lifecycle management introduces measurable throughput penalties (up to 37%) under rapid reader-set churn at low concurrency. Storage engine architects must evaluate workload churn rates and thread counts before adopting snapshot pinning over coarse-grained locking.

REFERENCES

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[2] J. Dean and S. Ghemawat, "LevelDB: A fast and lightweight key/value database library by Google," GitHub repository, google/leveldb, 2011.

[3] Meta Platforms Inc., "RocksDB: A persistent key-value store," GitHub repository, facebook/rocksdb, 2013.

[4] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Communications of the ACM*, vol. 33, no. 6, pp. 668–676, 1990.

[5] P. A. Bernstein and N. Goodman, "Concurrency control in distributed database systems," *ACM Computing Surveys*, vol. 13, no. 2, pp. 185–221, 1981.

[6] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "WiscKey: Separating keys from values in SSD-conscious storage," *ACM Transactions on Storage*, vol. 13, no. 1, article 5, 2017.

[7] N. Dayan, M. Athanassoulis, and S. Idreos, "Monkey: Optimal navigable key-value stores," in *Proceedings of the 2017 ACM SIGMOD International Conference on Management of Data*, 2017, pp. 79–94.

APPENDIX: ARTIFACTS AND REPRODUCIBILITY

All raw datasets, analysis scripts, and figure generators are open and reproducible:

- **Frozen Campaign Commits**: Protocol freeze at `a5c0fb97deb8a67705a80e76b49e25f31d692492`; 960-trial campaign completion at `e156b92`; 600-trial confirmatory campaign completion at `4a06217` (source code identical to `370fd6c5a138321ee3741bb7774e950a30d23587`).
- **Targeted Validation Executable Provenance**: The 600-trial confirmatory executable (`targeted_runner`, SHA-256: `2855b4318d5c574a2d0c3fbf5eb773f3ae7aebd1d0469961db3a9b8e6399832f`) was independently audited in Phase I6.10. Rebuilding from the native source tree on Go 1.27.3 yields a byte-for-byte exact SHA-256 and Go Build ID match. Because the execution directory lacked Git version metadata, raw trial records record `git_revision = "unknown_rev"`.
- **Raw Datasets**: 960-trial exploratory campaign at `research/experiments/data/campaign_wsl2_ext4_restart_v1/`; 600-trial confirmatory campaign at `research/experiments/data/targeted_validation_v1/`.
- **Checksum Manifests**: SHA-256 manifests provided for both datasets (`manifest_checksums.json`).
- **Analysis Scripts**: Full reproducibility package at `research/experiments/analysis/` and `research/reviews/`.